

PROJECTO DE Fundamentos da Programação

2º Semestre 2022/2023 (MEMec, LEAN)

Data limite para entrega: **Confirmar na página da cadeira**
17 de Junho de 2023, 23h59m (hora do sistema Fénix)

RESTAURANT ROBOT CLEANER

1. Introdução

O programa a realizar pretende simular uma versão simplificada de um *Restaurant robot cleaner*.¹ (omnidirecional). Este desloca-se num perímetro definido. A programação será incremental, começando com um projeto simples, indo incrementando a complexidade e realismo. Apresentam-se exemplos (mas não modelos) de sistemas semelhantes.



2. Requisitos do programa

O programa deverá utilizar a biblioteca `graphics` fornecida pelos autores do livro: *Python Programming: An Introduction to Computer Science, 3rd Ed., John M. Zelle, Franklin, Beedle & Associates, 2017*. Toda a interação será feita por via de interfaces gráficas. Deverá ser criada uma janela com um menu, onde os vários níveis de implementação descritos em seguida são escolhidos pelo utilizador e executados. Em cada um dos níveis o programa deverá sempre voltar ao menu.

2.1. Implementação básica (tier 1, Primeira implementação)

¹ Ver por exemplo:

https://www.gannett-cdn.com/presto/2022/06/09/PNJM/0d64b95d-dbe5-4505-a3b5-052a7ea609d2-baeb1a5d-b5e4-4e8d-af7c-9f76d810cc14_thumbnail.png?height=576

<https://eu.northjersey.com/story/news/2022/06/16/nj-restaurants-robot-waiter-sign-future-amid-labor-shortages/7551784001/>

<https://asia.nikkei.com/Business/Food-Beverage/Japan-s-Skylark-rolls-out-robo-waiters-for-contactless-dining>

A primeira implementação deverá conter uma classe obstáculo, chamada `Table`, que poderá utilizar as classes `Retangulo` e a classe `Circulo`, entre outras. A área de navegação deverá ser retangular e deverá ter no mínimo duas *docking station* encostados aos bordos.

O robot destina-se primariamente a manter a área de refeições limpas.

O utilizador poderá clicar em qualquer ponto na área de navegação onde a sujidade deverá ser eliminada. O robot deve sair da sua *docking station* e dirigir-se a esse ponto para o limpar. A limpeza deverá cobrir cerca do dobro da área do robot, estando o ponto clicado no centro da área a limpar. O movimento do robot pode ser circular, espiral, um quadrado simples ou outro considerado mais adequado. Após a limpeza, o robot deve colocar-se na *docking station* que for mais conveniente.

O robot pode ser implementado numa classe muito semelhante à classe `Projectil` dada nas aulas. A classe a criar deverá ter o nome `Waiter`. Esta classe deverá ter todos os métodos que se considerarem necessários e deverá utilizar todas as classes que se considerarem necessárias. Note-se que o robot choca com os obstáculos, ou seja, não pode sobrepor-se a esses objetos. A área de navegação, o robot e os obstáculos deverão ter tamanhos representativos do cenário proposto.

2.2. Implementação intermédia (tier 2, Segunda implementação)

Na segunda implementação a sala terá o número de obstáculos (Mesa, Cadeira ou outros) de qualquer dos formatos anteriores a definir pelo programador. A área deverá ter no mínimo 5 obstáculos, um dos quais deverá ser uma Mesa, circular ou oval, e não poderá estar encostado aos bordos. Deverá ser utilizada de novo a classe `Mesa`. O robot deverá **limpar a totalidade da sala** (preferencialmente minimizando o tempo...).

Uma possibilidade será o robot sair numa direção até ao extremo da sala, sofrer um incremento na outra direção e voltar para trás até chegar ao outro extremo da sala. Deste modo poderá cobrir toda a superfície num movimento de vaivém. Quando existir um obstáculo no caminho, o robot tem que o evitar (por exemplo contornar) o mesmo.

O utilizador deverá definir um conjunto de pontos (>1) de sujidade dados por *clicks* do rato. Ao passar por esses pontos a limpeza deve decorrer tal como na primeira implementação, e em seguida o robô deve retomar a trajetória de navegação da sala. Assuma que a orientação é efetuada a uma distancia suficiente para este poder rodar sem colisões.

O robot tem uma bateria que deve ser carregada após percorrer duas vezes a área de navegação. Terá de ser efetuada a gestão da bateria de forma a fazer uma pausa no serviço para voltar à *docking station* e carregar a bateria (a bateria é de carregamento super-rápido, demorando apenas 2 segundos). O robot deve ter uma luz indicadora da carga atual que muda de cor quando está a carregar. Poderá ainda mudar de cor quando a bateria estiver a esgotar-se. A posição do carregamento do robot deverá ser indicada como a *docking station* do robot.

2.3. Implementação intermédia (tier 3, Terceira implementação)

Na implementação final, a configuração da sala e obstáculos na sala deverá ser feita de duas formas:

- lida de um ficheiro (sala.txt);
- gerada aleatoriamente pelo programa (distribuição esparsa do objetos) – indicado como “aleat.ger”.

A escolha de uma ou outra forma será feita numa janela de menu, sendo indicado o nome correspondente. No caso de leitura de dados de um ficheiro, o robô deverá ler do ficheiro `Sala.txt` o tamanho da janela de simulação, seguido pelas entradas que definam a localização dos móveis na sala, em termos percentuais, relativos as dimensões desta. A localização da *docking station* é também definida pelo programa. O ficheiro deverá ter o seguinte formato (**o tamanho da janela e as localizações são apenas exemplificativos**):

```
Tamanho da janela de simulacao sugerida:
800 600
```

```
Localizacao dos objectos:
Dock Rectangle(Point(0,0), Point(5,5))
Mesa Rectangle(Point(20,40), Point(10,70))
Cadeira Circle(Point(50,50),5)
Mesa Rectangle(Point(90,10),Point(100,50))
...
```

Note que as localizações dos objetos deverão estar em percentagem, ou seja, tem de ser utilizada a instrução `win.setCoords(0.0,0.0,100.0,100.0)`. Após a leitura dos dados do ficheiro será criado o ambiente gráfico respetivo.

No caso de geração aleatória, a *docking station* é pré-definida pelo programa. O gerador aleatório define a localização dos móveis, onde as posições e formas serão aleatórias, mas garantindo que não há sobreposição e há espaço para o robô passar.

Os pontos de maior sujidade deverão ser introduzidos de duas formas nesta implementação:

- dados por um ficheiro;
- dados através de cliques do rato;

A escolha ente estes dois tipos de pedidos serão efetuados através de uma nova janela de menu. O ficheiro `Limpeza.txt` com os pedidos terá o seguinte formato:

```
Pontos de maior sujidade:
62.5 62.5
25.0 40.0
62.5 30.0
...
```

3. Indicações gerais e critérios de avaliação

- O trabalho deverá ser feito em grupos de dois alunos.
- O trabalho deve ser efetuado em **Python (3.9² ou anterior)**.

² (Instalação de defeito do Anaconda)

O projeto deverá ser entregue até ao dia (Ver página da disciplina / 1ª página), 23h59m (hora do sistema Fénix), constando de:

- documentação do trabalho (ver ponto 4);
- Todos os ficheiros do programa, (**incluindo graphics.py**) necessários
 - em bom estilo de programação
 - comentários esclarecedores do código,
 - paragrafação que facilite a leitura e sugira a estruturação do programa
 - nomes adequados para as variáveis, classes, métodos e funções;
- exemplos dos resultados da execução, mostrando claramente como o seu programa aborda todas as situações possíveis;
- listagem dos ficheiros com os dados requeridos;

Estes documentos deverão ser submetidos via Fénix num único ficheiro **.zip** ou **.rar.**, com o nome CP23-grxx.***

- todos os documentos bem como o código fonte deverão ser identificados com os nomes e números dos alunos que constituem o grupo. No caso dos documentos em formato **.pdf** a identificação é feita na capa, e no footer das páginas. No caso do código fonte essa identificação será feita em comentários internos.
- Haverá uma tolerância de 2 DIAS nas entregas com respetiva **PENALIZAÇÃO**. Os trabalhos entregues entre as 0h00min e as 23h59min do dia seguinte ao prazo estipulado **SOFRERÃO UMA PENALIZAÇÃO DE 2,5 VALORES**. Os trabalhos entregues entre as 0h00min e as 23h59min do 2º dia seguinte ao prazo estipulado **SOFRERÃO UMA PENALIZAÇÃO DE 5 VALORES**. Não serão aceites quaisquer trabalhos após o 2º dia.

4. Documentação

Todo o código fonte deverá incluir comentários apropriados, incluindo a identificação dos autores. A documentação do trabalho será dividida em dois tipos: a documentação destinada aos utilizadores do programa e a documentação técnica, destinada aos programadores que irão fazer a manutenção e possíveis alterações ao programa.

O — Generalidades:

- Capa, com a identificação do trabalho e dos elementos do grupo que realizaram o trabalho, data.
- Todas as folhas devem conter:
 - Header – com o nome do trabalho
 - Footer, numero de paginas/ total de páginas no exterior, e numero de grupo com elementos no interior (Gr-aaaaa-bbbbbb-cccc).
 - Texto em tamanho 12 justificado
 - Índice remissivo.

A — Sumário executivo (max 1 pagina)

Indicando explicitamente

- *As alíneas resolvidas*
- *As alíneas não funcionais*

B — Documentação para utilizadores. Esta deverá conter:

- a descrição do que o programa faz, incluindo a área geral de aplicação do programa e uma descrição do seu comportamento;
- a descrição do processo de utilização do programa (é boa ideia incluir um ou mais exemplos);
- a descrição da informação necessária para o bom funcionamento do programa (explicando por exemplo qual o formato correto de entrada de dados);
- a descrição da informação produzida pelo programa;
- a descrição, em termos não técnicos, das limitações do programa (deverão ser dados exemplos de situações em que o programa poderá ter interrupções inesperadas).

C — Documentação técnica. Esta deverá conter:

- a descrição da estrutura do programa incluindo as classes e os principais subprogramas (funções) e interligação entre eles, e contendo a descrição do algoritmo do programa principal e de cada subprograma, utilizando o aspeto gráfico do Capítulo 9 do livro;
- a descrição das estruturas de informação (de dados e de classes), justificando a sua escolha, e a descrição dos principais objetos, incluindo variáveis e instâncias das classes.

Repare que a documentação técnica **não** deve ser apenas uma listagem do programa acompanhada de comentários.

5. Critérios de avaliação

A nota do projeto será baseada nos seguintes aspetos:

Execução correta	60%	25%+15%+10%+10%
Facilidade de leitura	10%	
Documentação de utilização	10%	
Documentação técnica	10%	
Estilo de programação	10%	

Bonus/Malus

Penalizações / *Malus*: Além das referidas serão atribuídas penalizações (max 10%, 2 val.) por inconformidade relativamente às normas referidas, em particular,

- Nomes dos ficheiros requeridos
- Pobreza dos gráficos
- Formatação inadequada dos relatórios

poderão adicionar penalizações (até -10%, máx -2.0 val no total). O corpo docente é o único juiz da sua definição como tal.

Bonificações / *Bonus* : Refinamentos

- do desempenho,
- dos gráficos ou
- do algoritmo usado

poderão adicionar bonificações (até 10%, max 2.0 val no total, max 20 val no proj.). O corpo docente é o único juiz da sua aceitação como tal.

No item *Facilidade de leitura* serão avaliados os nomes das variáveis, classes, comentários, paragrafação e tamanho das classes. Qualquer função ou método com mais de 20 linhas será fortemente penalizado.

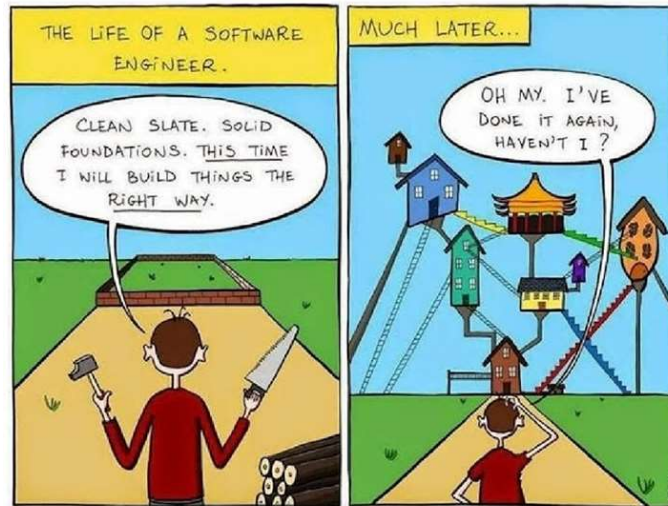
O *Estilo de programação* avalia a utilização de boas práticas de programação na elaboração do código fonte. São avaliados tópicos como:

- a existência de documentação interna ao código (cabeçalho, comentários explicativos ao longo do código, nomes de variáveis com significado, indentação do código para facilitar a leitura, etc.)
- modularidade (organização do código em classes de forma a facilitar a sua reutilização noutros programas, funções com argumentos de entrada e de saída de dados onde tal for adequado, etc.)
- **PODERÁ SER UTILIZADO UM SOFTWARE DE DETEÇÃO DE CÓPIAS. TODOS OS ALUNOS QUE SE PROVE TEREM COPIADO NOS PROJETO TERÃO A NOTA FINAL DE 0 (ZERO) VALORES, SENDO O FACTO PARTICIPADO AO CONSELHO PEDAGÓGICO DO IST.**

NOTA IMPORTANTE:

O corpo docente reserva-se o direito de convocar todos os alunos de um grupo a uma prova oral para defenderem a nota do trabalho apresentado.

BOM TRABALHO!



7



Notas sobre a implementação do robot

O modelo de implementação é o mais simples possível, e não são consideradas inércias. Considere um robot omnidirecional capaz de movimentos no plano, rodando em torno do centro de massa. O comando do(s) motor(es) é efetuado por deslocamento pela direção correspondente as direções transversais e longitudinais. Tratando-se de um modelo simplificado, dever-se-á tomar em atenção na implementação a possibilidade de uma eventual correção / desenvolvimento ou adaptação. As figuras seguintes apresentam exemplos robots omnidirecionais reais. Não é considerada neste trabalho a modelação do robot.

